

THOMAZ KLIFSON FALCÃO BARBOZA

**CBOM-DRIVEN CRYPTO-AGILITY FOR POST-QUANTUM MIGRATION IN
HYBRID THREE-TIER ENVIRONMENTS**

Method and experimental validation in a simulated banking topology

São Paulo
2026

THOMAZ KLIFSON FALCÃO BARBOZA

**CBOM-DRIVEN CRYPTO-AGILITY FOR POST-QUANTUM MIGRATION IN
HYBRID THREE-TIER ENVIRONMENTS**

Method and experimental validation in a simulated banking topology

Final Course Project submitted to the
Institute of Technology and Leadership
(INTELI), to obtain a bachelor's degree in
Computer Science.

Advisor: Prof. Reginaldo Arakaki

São Paulo
2026

Cataloging in Publication
Library and Documentation Service
Instituto de Tecnologia e Liderança (INTELI)
Data entered by the author.

BARBOZA, Thomaz Klifson Falcão. CBOM-driven crypto-agility for post-quantum migration in hybrid three-tier environments: method and experimental validation in a simulated banking topology. 2026. 31 leaves. Final Course Project (Bachelor's Degree in Computer Science) - Institute of Technology and Leadership, São Paulo, 2026.

RESUMO

A migração de criptografia de chave pública para algoritmos resistentes a adversários quânticos exige mais do que a substituição de primitivas. Em ambientes corporativos híbridos, dependências criptográficas aparecem em aplicações, protocolos, certificados, bibliotecas, bancos de dados, serviços de segredos e pipelines, o que dificulta inventário, priorização, implantação gradual e recuperação. Este trabalho propõe um roteiro de criptoagilidade orientado por Cryptography Bill of Materials (CBOM) para governar a migração em arquiteturas three-tier. O método combina inventário CBOM/SBOM, decisão por políticas, gateway de governança, implantação canária, observabilidade, critérios de rollback e retenção de evidências. A validação empregou uma topologia bancária simulada com oito nós lógicos em Podman rootless e cenários executados com k6. Nos testes nominais, S4-T01 concluiu 150 requisições sem falhas, com p95 de 39,71 ms, e S4-T02 concluiu 40 requisições sem falhas, com p95 de 24,70 ms. S4-T03 confirmou funcionalmente a cadeia aplicação, mock-vault, banco e nó canário por HTTP 200, sem comprovar mTLS. A comparação controlada S4-COMP manteve rota, carga, 5 usuários virtuais e 30 segundos, obtendo p95 de 42,58 ms no baseline e 27,20 ms no canário; esse resultado descreve diferenças operacionais, não overhead de criptografia pós-quântica. Sob stress, 10.000 requisições foram concluídas sem falhas HTTP, porém o p95 de 1149,64 ms violou o SLO LAT-01. Na indisponibilidade do mock-vault/KMS simulado, a aplicação retornou HTTP 500 e recuperou HTTP 200 após o reinício. As evidências indicam viabilidade do roteiro de governança e expõem limites relevantes: não foram comprovados mTLS real, KMS real ou algoritmos pós-quânticos no dataplane.

Palavras-chave: criptoagilidade; criptografia pós-quântica; CBOM; arquitetura three-tier; migração criptográfica.

ABSTRACT

Migrating public-key cryptography to algorithms designed to resist quantum adversaries requires more than replacing cryptographic primitives. In hybrid enterprise environments, cryptographic dependencies occur in applications, protocols, certificates, libraries, databases, secret services, and pipelines, making inventory, prioritization, gradual deployment, and recovery difficult. This work proposes a Cryptography Bill of Materials (CBOM)-driven crypto-agility roadmap for governing migration in three-tier architectures. The method combines CBOM/SBOM inventory, policy-driven decisions, a governance gateway, canary deployment, observability, rollback criteria, and evidence retention. Validation used a simulated banking topology with eight logical nodes running under rootless Podman and scenarios executed with k6. In nominal tests, S4-T01 completed 150 requests without failures and achieved a 39.71 ms p95, while S4-T02 completed 40 requests without failures and achieved a 24.70 ms p95. S4-T03 functionally confirmed the application, mock-vault, database, and canary chain through HTTP 200, without demonstrating mTLS. The controlled S4-COMP comparison kept the route, workload, five virtual users, and 30-second duration constant, obtaining p95 values of 42.58 ms for the baseline and 27.20 ms for the canary; this result describes operational differences, not post-quantum cryptography overhead. Under stress, 10,000 requests completed without HTTP failures, but the 1149.64 ms p95 violated the LAT-01 SLO. During simulated mock-vault/KMS unavailability, the application returned HTTP 500 and recovered to HTTP 200 after restart. The evidence indicates the feasibility of the governance roadmap while exposing relevant limits: real mTLS, a real KMS, and post-quantum algorithms in the dataplane were not demonstrated.

Key words: crypto-agility; post-quantum cryptography; CBOM; three-tier architecture; cryptographic migration.

LIST OF TABLES

Table 1 - Relationship between the problem, objectives, and evidence

Table 2 - Comparison of related approaches

Table 3 - Logical nodes in the experimental topology

Table 4 - Scenarios and evaluation criteria

Table 5 - Consolidated experimental results

Table 6 - Controlled baseline and canary comparison

Table 7 - Stress scenario results

Table 8 - Simulated mock-vault/KMS failure and recovery

Table 9 - Matrix of demonstrated and non-demonstrated claims

Table 10 - Validity threats and mitigation measures

LIST OF ABBREVIATIONS AND ACRONYMS

API - Application Programming Interface

CBOM - Cryptography Bill of Materials

CNG - Cryptography API: Next Generation

CSP - Cryptographic Service Provider

FIPS - Federal Information Processing Standards

HMAC - Hash-based Message Authentication Code

HNDL - Harvest Now, Decrypt Later

HSM - Hardware Security Module

HTTP - Hypertext Transfer Protocol

JCA - Java Cryptography Architecture

KEM - Key-Encapsulation Mechanism

KMS - Key Management Service

mTLS - Mutual Transport Layer Security

NCCoE - National Cybersecurity Center of Excellence

NIST - National Institute of Standards and Technology

PQC - Post-Quantum Cryptography

SBOM - Software Bill of Materials

SLO - Service Level Objective

TLS - Transport Layer Security

VU - Virtual User

Summary

1 INTRODUCTION.....	9
1.1 Context and motivation.....	9
1.2 Research problem.....	9
1.3 General objective.....	9
1.4 Specific objectives.....	10
1.5 Rationale.....	10
1.6 Scope.....	10
1.7 Realized contributions.....	12
2 THEORETICAL FOUNDATION.....	13
2.1 Crypto-agility.....	13
2.2 Post-quantum cryptography.....	13
2.3 Harvest now, decrypt later risk.....	13
2.4 CBOM and SBOM.....	14
2.5 Hybrid three-tier environments.....	14
2.6 NIST, NCCoE, and FIPS 203, 204, and 205.....	14
2.7 Cryptographic service providers.....	14
3 RELATED WORK.....	16
3.1 Microsoft CryptoAPI/CSP and CNG.....	16
3.2 Java Cryptography Architecture.....	16
3.3 PKCS #11 and HSMs.....	16
3.4 NIST/NCCoE Migration to Post-Quantum Cryptography.....	16
3.5 CycloneDX/CBOM.....	16
3.6 IBM/PQCA CBOMkit.....	16
3.7 Academic research.....	17
3.8 Novelty synthesis.....	18
4 METHODOLOGY.....	19
4.1 Research type.....	19
4.2 Methodological approach.....	19
4.3 Experimental topology.....	19
4.4 S4 scenarios.....	20
4.5 Metrics and criteria.....	21
4.6 Reproducibility protocol.....	21
4.7 Ethics and data protection.....	22
5 ARCHITECTURE AND IMPLEMENTATION.....	23
5.1 Overview.....	23
5.2 CBOM flow.....	23
5.3 Gateway and policy decisions.....	23
5.4 Canary deployment.....	23
5.5 Simulated mock-vault/KMS.....	23
5.6 Manifest integrity with HMAC-SHA256.....	24

5.7 Observability, load, and chaos.....	24
6 EXPERIMENTS AND RESULTS.....	25
6.1 Consolidated view.....	25
6.2 S4-T01 baseline.....	25
6.3 S4-T02 canary.....	25
6.4 S4-T03 app-chain.....	25
6.5 S4-COMP baseline versus canary.....	26
6.6 S4-E03 stress.....	26
6.7 S4-E01 simulated mock-vault/KMS unavailable.....	27
6.8 S4-E04 manifest integrity.....	27
7 DISCUSSION.....	29
7.1 Interpretation of results.....	29
7.2 What was demonstrated.....	29
7.3 Providing primitives versus governing migration.....	30
7.4 Comparison with related work.....	30
7.5 Healthchecks and observability.....	30
8 LIMITATIONS AND THREATS TO VALIDITY.....	31
8.1 Limitations.....	31
8.2 Threats to validity.....	31
9 CONCLUSION.....	33
REFERENCES.....	34
APPENDIX A - EXPERIMENTAL EVIDENCE.....	36
APPENDIX B - REPRODUCTION COMMANDS.....	37
APPENDIX C - ARTIFACT STRUCTURE.....	38
APPENDIX D - PENDING ITEMS FOR THE FINAL VERSION.....	39

1 INTRODUCTION

1.1 Context and motivation

Public-key cryptography supports authentication, key establishment, digital signatures, and communication protection in enterprise systems. The possibility that cryptographically relevant quantum computers may compromise the mathematical problems used by RSA and elliptic-curve cryptography increases the need to transition to post-quantum algorithms. Urgency does not depend only on when such a computer will become available. Data captured today may remain sensitive for years and be stored for future decryption, characterizing the *harvest now, decrypt later* risk (MOSCA, 2018; NCCOE, 2023).

On August 13, 2024, NIST published the first post-quantum standards: FIPS 203 for ML-KEM, FIPS 204 for ML-DSA, and FIPS 205 for SLH-DSA (NIST, 2024a; 2024b; 2024c). The existence of standards, however, does not make migration automatic. An organization must discover where algorithms and keys are used, understand dependencies across tiers, prioritize assets, prepare compatibility, and execute changes without disrupting critical services. NIST defines crypto-agility as the capabilities required to replace and adapt algorithms in protocols, applications, software, hardware, firmware, and infrastructure while preserving security and ongoing operations (BARKER et al., 2025).

Hybrid three-tier environments make this problem particularly distributed. The web tier may terminate TLS; the application tier may use libraries, certificates, and APIs; the data tier may depend on encryption at rest, backups, and secret services. On-premises, cloud, and legacy components may have different update cycles and owners. A migration without inventory and evidence may fix one point while leaving others vulnerable, or introduce incompatibility, latency regressions, and unavailability.

1.2 Research problem

The investigated problem is the lack of an integrated operational flow connecting cryptographic discovery, policy-based prioritization, incremental deployment, observability, and recovery in distributed enterprise architectures.

The research question is:

How can a CBOM-driven roadmap support the governance and incremental validation of post-quantum cryptographic migration in a hybrid three-tier environment while preserving traceability and explicitly reporting operational limits?

1.3 General objective

To develop and experimentally validate a reproducible CBOM-driven crypto-agility roadmap for governing cryptographic migration in hybrid three-tier environments.

1.4 Specific objectives

- 1 Systematize the concepts of crypto-agility, post-quantum cryptography, CBOM/SBOM, and three-tier architecture.
- 2 Distinguish the provision of cryptographic primitives from migration governance in distributed systems.
- 3 Define a flow connecting inventory, policies, decisions, canary deployment, observability, and rollback.
- 4 Build a simulated banking topology with eight logical nodes and synthetic data.
- 5 Execute nominal, controlled comparison, stress, dependency-failure, and manifest-integrity scenarios.
- 6 Evaluate availability, latency, recovery, and traceability without exceeding what the artifacts demonstrate.

1.5 Rationale

The practical contribution arises from the difficulty of transforming guidance and standards into a verifiable operational process. Cryptographic providers, libraries, and HSMs make primitives and keys accessible, but they do not automatically produce an organizational inventory or define priorities, canaries, SLOs, and rollback. CBOMs improve visibility, but representing an asset does not determine when and how to migrate it. This work integrates those elements into an observable and reproducible roadmap.

The academic contribution lies in combining and evaluating components that are often studied separately. Hasan et al. (2023) propose dependency analysis to prioritize migration; Näther et al. (2024) show that definitions of crypto-agility still vary; Cho et al. (2024) discuss centralized governance through software-defined cryptography. This study adds local validation in a three-tier topology, including controlled comparison, stress, and dependency unavailability, while preserving negative results in the analysis.

1.6 Scope

The scope includes:

- CBOM/SBOM inventory and dependency representation;

- a policy-decision gateway prototype;
- simulated canary deployment and validation;
- an eight-node logical topology under rootless Podman;
- k6 load testing;
- failure and recovery of a simulated mock-vault/KMS;
- manifest integrity with HMAC-SHA256;
- retention of logs, JSON files, command outputs, and consolidated results.

The scope excludes:

- production or distributed cloud deployment;
- real mTLS between nodes;
- a real KMS or HSM;
- demonstrated execution of ML-KEM, ML-DSA, or SLH-DSA in the dataplane;
- measurement of post-quantum cryptographic overhead;
- fully automated rollback;
- cost analysis;

- expert interviews or research involving human participants.

1.7 Realized contributions

The realized contributions are:

- 1 a five-stage roadmap: discover, decide, migrate, validate, and consolidate;
- 2 a technical distinction between cryptographic service providers and migration governance;
- 3 a reproducible eight-node topology covering application, data, security, control, observability, and load roles;
- 4 a revalidated experimental campaign including favorable outcomes and SLO violations;
- 5 an evidence package enabling audit of reported values;
- 6 an explicit analysis of what was and was not demonstrated.

Table 1 – Relationship between the problem, objectives, and evidence

Element	Formulation	Primary evidence
Problem	Distributed dependencies hinder migration governance	CBOM, topology, and S4 scenarios
Objective	Validate a CBOM-driven roadmap	gateway, canary, and evidence
Nominal performance	Verify operation under nominal load	S4-T01, S4-T02, and S4-T03
Comparability	Control route, workload, and duration	S4-COMP
Resilience	Observe stress and dependency failure	S4-E03 and S4-E01
Integrity	Block a tampered manifest	complementary/local S4-E04; server run pending

Source:** prepared by the authors (2026).

2 THEORETICAL FOUNDATION

2.1 Crypto-agility

Crypto-agility is the ability to adapt or replace cryptographic mechanisms without causing disproportionate application changes or disrupting operations. The concept encompasses algorithms, protocols, libraries, interfaces, hardware, firmware, and management processes. The review by Näther et al. (2024) identifies multiple dimensions in existing definitions, including replaceability, interoperability, and organizational context. NIST emphasizes that crypto-agility is not merely a code property: it requires coordination among architects, developers, operators, risk managers, and suppliers (BARKER et al., 2025).

An architecture may allow an algorithm to be changed through configuration and still not be crypto-agile in the organizational sense. If the organization does not know which services use the algorithm, cannot measure compatibility, or lacks rollback, replacement remains risky. In this work, crypto-agility is operationalized through inventory, policy-driven decisions, limited deployment, SLO observation, and evidence retention.

2.2 Post-quantum cryptography

Post-quantum cryptography comprises algorithms that run on conventional computers and are designed to resist known classical and quantum attacks. FIPS 203 specifies ML-KEM for establishing a shared secret. FIPS 204 specifies ML-DSA, a lattice-based signature family. FIPS 205 specifies SLH-DSA, a stateless hash-based signature scheme (NIST, 2024a; 2024b; 2024c).

The standards define primitives, parameters, and operations. By themselves, they do not specify how an organization should locate RSA certificates, replace third-party integrations, update protocols, observe regressions, or coordinate multiple environments. The work by Bos et al. (2018), concerning the precursor to ML-KEM, illustrates that primitive security and performance are a distinct analytical level from governance of adoption.

2.3 Harvest now, decrypt later risk

HNDL occurs when an adversary collects encrypted traffic today and retains it until cryptanalytic or computational advances allow decryption. Mosca's model (2018) relates the confidentiality lifetime of data, the time required for migration, and the threat horizon. Therefore, assets with long secrecy requirements may require action before a quantum computer capable of breaking current algorithms exists.

This risk also explains why migration must begin with inventory. Without knowing where long-lived data is protected and which algorithms participate in key establishment, the organization cannot prioritize effectively.

2.4 CBOM and SBOM

An SBOM records software components and their relationships. A CBOM complements that view with algorithms, protocols, certificates, keys, cryptographic libraries, parameters, and relationships among those assets. CycloneDX provides a structured representation for cryptographic assets and their dependencies, supporting transparency and preparation for post-quantum transition (OWASP CYCLONEDX, 2026).

CBOMkit combines CBOM generation, visualization, storage, and compliance checking. Its documentation also acknowledges limitations in static coverage, especially when a project is not built before scanning (CBOMKIT PROJECT, 2026). This reinforces that automated inventory should be validated and combined with runtime configuration, deployed certificates, and telemetry.

2.5 Hybrid three-tier environments

Three-tier architectures separate presentation, application logic, and persistence. In a hybrid environment, each tier may be distributed across local and cloud infrastructure, with gateways, internal networks, managed services, and legacy components. Cryptography crosses these boundaries: TLS at ingress, authentication and tokens in the application, database connections, encryption at rest, backups, and secret management.

Tier separation supports responsibility boundaries but also creates end-to-end dependencies. A key-establishment change may affect the client, proxy, service, and library. Migration should therefore observe the full chain rather than only the component implementing a primitive.

2.6 NIST, NCCoE, and FIPS 203, 204, and 205

The NCCoE post-quantum migration project emphasizes public-key cryptography discovery and interoperability testing (NCCOE, 2023a; 2023b). These materials guide organizations and present reference architectures, but they must be adapted to local contexts. This roadmap uses discovery as input, associates policies with it, and adds its own canary, load, and failure experiments.

FIPS 203, 204, and 205 are treated as normative target states, not implementation evidence. No experimental result in this work demonstrates that those primitives were executed in the dataplane.

2.7 Cryptographic service providers

Cryptographic providers solve access to implementations and cryptographic material. In the legacy Windows model, a Cryptographic Service Provider contains implementations of standards and algorithms and may operate in software or delegate to hardware. Microsoft recommends CNG as an extensible, cryptography-agnostic replacement for CryptoAPI/CSP (MICROSOFT, 2021; 2025).

The Java Cryptography Architecture uses service classes and pluggable providers for signatures, hashes, certificates, encryption, key generation and management, random numbers, and key-encapsulation mechanisms. Applications may request a service without binding to a specific implementation (ORACLE, 2025). PKCS #11 defines an interface for cryptographic tokens and is widely used to access HSMs and smart cards (OASIS OPEN, 2023). HSMs can protect keys and execute operations inside modules with specific requirements; FIPS 140-3 defines security requirements for cryptographic modules (NIST, 2019).

The abstraction provided by these technologies is relevant to crypto-agility because it reduces coupling between consumers and implementations. However, providing primitives is not equivalent to governing migration. A CSP, JCA, or HSM does not automatically discover every organizational usage point, classify business criticality, select migration waves, deploy inter-service canaries, or independently preserve an integrated decision, SLO, and recovery trail.

3 RELATED WORK

3.1 Microsoft CryptoAPI/CSP and CNG

CryptoAPI/CSP is an indirect competitor because it demonstrates cryptographic service abstraction at the operating-system level. Its limitation for this research problem is scope: it provides operations and providers, but not an organizational process for inventory and distributed migration. CNG improves extensibility and key management, yet remains cryptographic execution infrastructure rather than a complete governance cycle.

3.2 Java Cryptography Architecture

JCA provides implementation independence, provider interoperability, and algorithm extensibility (ORACLE, 2025). These properties may reduce replacement cost inside a Java application. JCA, however, does not represent dependencies across heterogeneous services, external certificates, proxies, databases, and cloud environments. It is a potential dataplane component, while this proposal operates in the governance plane.

3.3 PKCS #11 and HSMs

PKCS #11 and HSMs protect cryptographic material and standardize token operations. They are essential for custody and reliable key execution, but do not replace CBOM, risk analysis, or canary validation. This work did not use a real HSM; the mock-vault must not be confused with a PKCS #11 token or a FIPS 140-3 validated module.

3.4 NIST/NCCoE Migration to Post-Quantum Cryptography

NCCoE is the closest approach in process guidance because it relates discovery, architecture, and standards testing (NCCOE, 2023a; 2023b). The difference is that national guidance does not replace local implementation. This work materializes a reduced version of the cycle in a three-tier topology and records specific outcomes, including violations.

3.5 CycloneDX/CBOM

CycloneDX provides the representation language. Its strength is interoperability and detailed asset modeling. Its limitation is that it does not execute decisions or rollout. A CBOM may identify RSA or ECDH, but it does not independently determine business risk, migration sequence, or the response to a violated SLO.

3.6 IBM/PQCA CBOMkit

CBOMkit is a direct competitor in discovery and analysis. Originating in IBM cryptographic-inventory work and now maintained as an open project, it generates, visualizes, stores, and evaluates CBOMs, including external policies (IBM RESEARCH, 2022; CBOMKIT PROJECT, 2026). Its focus, however, does not

independently cover the full gateway, canary, stress, dependency failure, and recovery chain evaluated here. This proposal uses the inventory idea and adds an operational roadmap.

3.7 Academic research

Hasan et al. (2023) structure migration through dependency analysis and case studies, contributing to prioritization. Näther et al. (2024) systematize definitions of crypto-agility and demonstrate the need for conceptual precision. Cho et al. (2024) propose software-defined cryptography and centralized governance. These works support the need for abstraction, inventory, and policy, but do not present the same experimental combination in a three-tier topology with the S4 scenarios executed here.

Table 2 – Comparison of related approaches

Approach	Primary focus	CBOM inventory	Policy decisions	Canary validation	Stress/failure	Recovery/rollback	Limitation relative to the proposal
CryptoAPI /CSP and CNG	Windows APIs, algorithms, and keys	No	Partial, by configuration	No	No	No	Does not govern distributed dependencies
JCA	Java cryptographic providers and services	No	Partial, by provider selection	No	No	No	Limited to the Java application/runtime
PKCS #11/HSM	Tokens, custody, and protected operations	No	Partial	No	No	Partial, for keys	Does not prioritize organizational migration
NIST/NCCoE	Guidance, discovery, and interoperability	Yes, as input	Guidance	Test guidance	Partial	Guidance	Does not replace local implementation
CyclonedX/CBOM	Cryptographic asset representation	Yes	No	No	No	No	Represents but does not orchestrate

CBOMkit	Discovery, visualization, and compliance	Yes	Yes, limited/extendable	No	No	No	Does not cover the entire operational cycle
Academic literature	Models, dependencies, and concepts	Variable	Variable	Rare	Variable	Variable	Three-tier evidence remains limited
Proposal	Integrated governance and experimental evidence	Yes	Yes	Yes	Yes	Partial	Simulated environment and no PQC dataplane

Source:** prepared by the authors based on Microsoft (2021; 2025), Oracle (2025), OASIS Open (2023), NCCoE (2023a; 2023b), OWASP CycloneDX (2026), CBOMkit Project (2026), Hasan et al. (2023), Näther et al. (2024), and Cho et al. (2024).

3.8 Novelty synthesis

The novelty is not a new primitive or a replacement for CSPs. It is the organization of a higher layer that governs transitions between distributed cryptographic states. This layer uses CBOM as a source, policies for prioritization, a gateway for recording decisions, canary deployment for limiting impact, SLOs for validation, evidence for audit, and recovery procedures. The combination brings normative recommendations closer to a reproducible and measurable process.

4 METHODOLOGY

4.1 Research type

This is applied and exploratory research conducted as a quantitative case study in a controlled environment. It is applied because it produces and evaluates a roadmap and prototypes; it is exploratory because it investigates how to integrate governance components that are not yet consolidated in a specific topology.

Earlier project documents proposed interviews and a cloud environment. These stages have no final evidence in the repository and therefore are not part of the executed method described in this thesis.

4.2 Methodological approach

The study was organized into six stages:

- 1 review standards, guidance, and literature;
- 2 define requirements and metrics;
- 3 model the eight-node topology;
- 4 implement CBOM and gateway prototypes;
- 5 execute and revalidate the S4 scenarios;
- 6 triangulate JSON summaries, logs, HTTP responses, and documentation.

The unit of analysis was the three-tier chain and its governance controls during nominal operation, controlled comparison, stress, dependency failure, and manifest tampering.

4.3 Experimental topology

The environment ran on an academic Ubuntu 24.04.3 LTS server with 64 vCPUs, 125 GiB of RAM, and rootless Podman 4.9.3. Using Podman without a daemon or elevated privileges reduces dependence on administrative access, but also differs from a production cluster (PODMAN PROJECT, 2026).

Table 3 – Logical nodes in the experimental topology

Logical node	Role	Main tiers/networks
k8s-control	control plane and prototype execution	cluster and observability

k8s-worker-1	web ingress and baseline	ingress, cluster, and observability
k8s-worker-2	application and /chain flow	cluster, data, security, and observability
k8s-worker-3	replica marked as canary	ingress, cluster, data, security, and observability
data-node	PostgreSQL and synthetic events	data and observability
security-integration-node	HTTP simulated mock-vault/KMS	security and observability
observability-node	Prometheus planned for metrics	observability
load-chaos-node	k6 and failure induction	ingress and observability

Source:** prepared by the authors from project artifacts (2026).

4.4 S4 scenarios

Table 4 – Scenarios and evaluation criteria

Scenario	Category	Configuration	Primary criterion
S4-T01	nominal	baseline, 5 VUs, 30 s	0 failures and LAT-01
S4-T02	nominal	canary, short nominal load	0 failures and LAT-01
S4-T03	functional	app, security, database, and canary chain	HTTP 200 and functional fields
S4-COMP	comparison	same route, 5 VUs, 30 s, 150 requests per node	descriptive comparison
S4-E03	stress	/chain, 50 VUs, 10,000 iterations	availability and LAT-01
S4-E01	failure	stop and restart of security node	detection, tolerance, and recovery
S4-E04	complementary/local integrity	intact and tampered manifest	prototype blocking; server reexecution pending

Source:** prepared by the authors (2026).

4.5 Metrics and criteria

The quantitative metrics were:

- completed requests and iterations;
- HTTP failure rate;
- mean latency;
- 95th-percentile latency;
- maximum stress latency;
- HTTP status before, during, and after failure;
- database event counter;
- integrity-verification result.

LAT-01 requires p95 below 150 ms. Functional availability means completion with a successful HTTP status. Recovery means return to HTTP 200 after restoring the dependency. These criteria do not measure post-quantum resistance because there were no PQC primitives in the dataplane.

4.6 Reproducibility protocol

Revalidated evidence is under `docs/revalidation/` and in the package located at the repository root, `revalidation-evidence-2026-06-10.tgz`. The protocol requires:

- 1 record host, Podman, image, and k6 versions;
- 2 validate the topology before load generation;
- 3 run the scenario-specific script;
- 4 preserve textual output and JSON summary;
- 5 record manual failure and recovery actions;

- 6 compare extracted values with the published table;
- 7 do not modify original logs;
- 8 in future studies, run at least three independent repetitions and calculate dispersion and confidence intervals.

4.7 Ethics and data protection

The study used synthetic data and a simulated topology. No real banking data was processed and no interviews were conducted. Therefore, no personal data from participants was collected. Future deployments involving data or specialists should follow the applicable ethical and institutional procedures.

5 ARCHITECTURE AND IMPLEMENTATION

5.1 Overview

The architecture separates the dataplane and governance plane. The simulated dataplane contains web ingress, application, database, and security integration. The governance plane contains inventory, policies, gateway, evidence, and deployment procedures. This separation avoids claiming that the gateway executes post-quantum cryptography: it organizes decisions about cryptographic states.

5.2 CBOM flow

The scanner produces manifests containing components and findings. The gateway ingests the CBOM, validates structure and integrity, classifies algorithms according to policies, and generates recommended actions. In the evaluated prototype, a manifest containing three components produced two high-risk findings, two violations, and two migration actions.

A CBOM is an observable representation, not an infallible source. Static findings may be incomplete, and runtime configurations may differ from source code. The roadmap therefore anticipates later correlation with telemetry and deployment inventory.

5.3 Gateway and policy decisions

The prototype gateway represents six conceptual responsibilities: CBOM ingestion, policy evaluation, orchestration, telemetry correlation, rollback/chaos, and governance integration. In the available implementation, the most concrete functions are ingestion, classification, recommendation, and manifest integrity. Integrations with queues, GitOps, service mesh, and SIEM remain proposed architecture.

Policies decouple migration rules from business code. They may declare forbidden algorithms, target states, criticality, and exceptions. Each decision should be versioned and linked to the input CBOM for auditability.

5.4 Canary deployment

`k8s-worker-3` represents a canary replica. Canary validation reduces the exposed population before expanding a change. In this environment, the canary label is operational; there is no evidence that the node runs a different post-quantum cryptographic stack. S4-COMP compares nodes under the same conditions but does not measure cryptographic overhead.

5.5 Simulated mock-vault/KMS

`security-integration-node` exposes a static profile identified as `mock-vault` over HTTP. It supports chain and dependency-unavailability testing, but does not perform real key generation, rotation, envelope encryption, or custody. Therefore, KMS is always qualified as “simulated.”

5.6 Manifest integrity with HMAC-SHA256

The complementary/local S4-E04 evidence revealed that manifest tampering could reduce recommended actions without blocking automation. The prototype was modified to calculate HMAC-SHA256 over a canonical manifest representation and verify the value before processing. An invalid artifact now produces exit code 1.

HMAC-SHA256 provides symmetric authentication if the key remains protected. The implementation uses an environment key and local samples; it is not equivalent to enterprise digital signatures, PKI, HSM, or complete key management. Repository documents confirm implementation and local testing, but still list final reexecution on the academic server as pending.

5.7 Observability, load, and chaos

k6 generates workloads and records HTTP metrics. Application logs and database counters complement observation. The Prometheus/Grafana stack was planned, but `observability-node` failed in runtime captures and several healthchecks appeared as `unhealthy` even when functional endpoints responded. Observability must therefore be considered partial.

6 EXPERIMENTS AND RESULTS

6.1 Consolidated view

Table 5 consolidates only the six principal revalidation scenarios contained in the June 10, 2026 evidence package. S4-E04 is presented separately in Section 6.8 as complementary/local prototype evidence.

Table 5 – Consolidated experimental results

Scenario	Observed evidence	Status
S4-T01	150 requests and iterations; 0 failures; mean 30.36 ms; p95 39.71 ms	approved
S4-T02	40 requests and iterations; 0 failures; mean 20.43 ms; p95 24.70 ms	approved
S4-T03	HTTP 200; 51.17 ms; mock-vault; database OK; functional chain	functionally approved
S4-COMP	150 + 150 requests; 0 failures; p95 42.58 ms and 27.20 ms	approved as operational comparison
S4-E03	10,000 requests; 0 failures; p95 1149.64 ms	availability approved; LAT-01 failed
S4-E01	HTTP 200, then 500, then 200	partially approved

Source:** prepared by the authors from the evidence package (2026).

6.2 S4-T01 baseline

S4-T01 executed 150 iterations and 150 requests against /chain. No HTTP failures were recorded. Mean latency was 30.36 ms and p95 was 39.71 ms, below the 150 ms LAT-01 limit. The scenario was approved for the observed nominal window.

6.3 S4-T02 canary

S4-T02 executed 40 iterations and 40 requests against the node marked as canary. There were no HTTP failures. Mean latency was 20.43 ms and p95 was 24.70 ms. The scenario was approved, but its different workload prevents a direct causal comparison with S4-T01.

6.4 S4-T03 app-chain

The call traversed the application, security-integration-node, database, and canary node. The response was HTTP 200 with `elapsed_ms = 51.17`, `security.secret_profile`

= mock-vault, and database.ok = true. The result demonstrates functional chaining. It does not demonstrate mTLS because the code and logs show access to `http://security-integration-node:8080/secret`.

6.5 S4-COMP baseline versus canary

S4-COMP was created to address the methodological limitation of comparing S4-T01 and S4-T02. Both nodes received the same `/chain` route, 5 VUs, a 30-second duration, and 150 requests.

Table 6 – Controlled baseline and canary comparison

Metric	Baseline	Canary	Variation
VUs	5	5	-
Duration	30 s	30 s	-
Requests	150	150	-
HTTP failures	0	0	-
Mean	31.13 ms	19.87 ms	-36.19%
p95	42.58 ms	27.20 ms	-36.12%

Source:** prepared by the authors from k6 summaries (2026).

The canary showed lower latency in this execution. The difference cannot be attributed to PQC or mTLS because neither condition was demonstrated in the dataplane. The result is an operational comparison between two instances in the simulated topology.

6.6 S4-E03 stress

The scenario executed 10,000 synthetic transactions against `/chain` with 50 VUs. All requests completed and there were no HTTP failures.

Table 7 – Stress scenario results

Metric	Result
Completed requests	10,000
Iterations	10,000
HTTP failures	0
Failure rate	0.00%

Mean	203.47 ms
p95	1149.64 ms
Maximum	6235.52 ms
Functional availability	approved
LAT-01, p95 < 150 ms	failed

Source:** prepared by the authors from S4-E03-stress/k6-summary.json (2026).

The architecture maintained functional availability under compressed load, but tail latency degraded substantially. The p95 was approximately 7.66 times the 150 ms limit. The result identifies a bottleneck or contention that the available instrumentation could not localize.

6.7 S4-E01 simulated mock-vault/KMS unavailable

Table 8 – Simulated mock-vault/KMS failure and recovery

Phase	Service state	HTTP	Client latency	Evidence
Before	available	200	40.09 ms	mock-vault, database OK, event_count=115 00
During	node stopped	500	18.21 ms	<urlopen error [Errno -2] Name or service not known>
After	node restarted	200	36.25 ms	mock-vault, database OK, event_count=115 01

Source:** prepared by the authors from the three S4-E01 captures (2026).

The scenario passed failure detection and post-restart recovery but failed tolerance without an HTTP error. The application did not maintain a successful response during unavailability. The shorter HTTP 500 latency is not an improvement; the chain failed before completing its dependencies.

6.8 S4-E04 manifest integrity

The first tampering test showed that removing a finding reduced recommended actions from two to one without blocking. After HMAC-SHA256 was added, a manifest with an invalid authenticator was rejected before automation. The implementation and samples are in the repository, and the local test is documented.

S4-E04 is not part of the principal June 10, 2026 revalidation or its evidence package. The project checklist still marks final server reexecution and log archival as pending. Therefore, the conclusion is limited to the local prototype.

7 DISCUSSION

7.1 Interpretation of results

Nominal scenarios indicate that the topology can execute the functional chain with low latency during the evaluated windows. S4-COMP improves descriptive validity by controlling route, duration, concurrency, and sample size. Even so, one execution cannot establish causality for the canary label.

S4-E03 demonstrates that availability and performance are separate dimensions. A 0% error rate did not prevent violation of LAT-01. For banking systems sensitive to tail latency, this result is material and prevents a production-readiness conclusion.

S4-E01 demonstrates simple detection and recovery, but not full tolerance. Propagation of HTTP 500 suggests the absence of secure caching, redundancy, a circuit breaker with controlled degradation, or another strategy compatible with business requirements. Any fallback involving a security dependency must be carefully analyzed to avoid weakening controls.

7.2 What was demonstrated

Table 9 – Matrix of demonstrated and non-demonstrated claims

Claim	Status	Basis
The topology responds in nominal baseline and canary tests	demonstrated in the windows	S4-T01 and S4-T02
The app, mock-vault, database, and canary chain works	demonstrated through HTTP	S4-T03
Baseline and canary can be compared under equal load	descriptively demonstrated	S4-COMP
10,000 requests complete without HTTP failure	demonstrated	S4-E03
LAT-01 is violated under stress	demonstrated	S4-E03
Mock-vault failure causes error and restart restores service	demonstrated	S4-E01
HMAC can block the tampered sample	complementary/local evidence; server run pending	S4-E04

Source:** prepared by the authors (2026).

Real mTLS, a real KMS, HSM use, PQC in the dataplane, PQC/mTLS overhead, high availability during failure, automated rollback, complete observability, and production readiness were not demonstrated.

7.3 Providing primitives versus governing migration

CSP, CNG, JCA, and PKCS #11 provide stable interfaces for requesting operations and selecting implementations. HSMs add physical and logical protection for keys and operations. These mechanisms may be used by the target architecture and facilitate replacement within a technical context.

Migration governance requires a different decision scale. Consumers must be discovered, dependencies correlated, data and services classified, policy defined, waves selected, manifests validated, canaries executed, SLOs observed, expansion stopped when needed, evidence recorded, and state recovered. The proposal adopts the decoupling idea of providers but is positioned above them. It may decide that a service should migrate and track the change; execution of the primitive would remain the responsibility of a library, provider, token, or cryptographic service.

This distinction strengthens the novelty by avoiding the incorrect claim that the proposal is a complete CSP. The work integrates existing components into a governance cycle and provides local experimental evidence. Its contribution is architectural and methodological, not algorithmic.

7.4 Comparison with related work

NIST/NCCoE offers broader normative and architectural coverage but does not automatically generate evidence for a specific organization. CycloneDX and CBOMkit provide more mature inventory capabilities than the local prototype, but do not replace the canary, stress, and recovery process. Hasan et al. (2023) offer more formal dependency analysis; Cho et al. (2024) provide deeper treatment of software-defined governance. This proposal contributes by bringing these elements together in a reproducible campaign and publishing outcomes that oppose an exclusively positive narrative.

7.5 Healthchecks and observability

Inconsistent healthchecks are an operational limitation. Functional endpoints responded even when the runtime marked containers as `unhealthy`. This may result from healthcheck definitions, the Compose provider, or initialization order. The divergence should not be ignored: in production, incorrect healthchecks may cause restarts, improper traffic removal, or hidden failures. Prometheus/Grafana and healthchecks must be corrected before drawing further conclusions about latency causes.

8 LIMITATIONS AND THREATS TO VALIDITY

8.1 Limitations

- 1 The environment is simulated and runs on a single academic server.
- 2 Rootless Podman does not reproduce every behavior of Kubernetes or enterprise cloud.
- 3 The mock-vault is not a real KMS and does not use an HSM.
- 4 S4-T03 uses HTTP and does not demonstrate mTLS.
- 5 No ML-KEM, ML-DSA, or SLH-DSA execution was demonstrated in the dataplane.
- 6 S4-COMP does not measure PQC or mTLS overhead.
- 7 S4-E03 violated LAT-01.
- 8 S4-E01 returned HTTP 500 during failure.
- 9 Healthchecks and observability are inconsistent.
- 10 Most scenarios had one revalidated execution, without statistical analysis.
- 11 S4-E04 has local evidence but lacks the final server reexecution listed in the checklist.
- 12 Automated rollback, cost, real rotation, and expired-certificate scenarios remain pending.

8.2 Threats to validity

Table 10 – Validity threats and mitigation measures

Type	Threat	Applied or future mitigation
Construct	HTTP latency does not measure PQC security	separate operational metrics from cryptographic claims
Construct	historical names suggest mTLS/KMS	interpret observed protocol and code

Internal	shared host, cache, and scheduling	record environment; repeat and randomize later
Internal	S4-E01 failure was induced manually	preserve timestamps and commands
Conclusion	single execution and no confidence interval	restrict analysis to descriptive statistics
External	eight containers do not represent a real bank	do not extrapolate to production
External	no cloud, HSM, or service mesh	expand the environment in future work
Reproducibility	latest images may change	pin versions and digests in the next campaign

Source:** prepared by the authors (2026).

9 CONCLUSION

This work developed a practical CBOM-driven crypto-agility roadmap for hybrid three-tier environments. The proposal connects discovery, policy-driven decisions, canary deployment, observability, recovery, and evidence retention. Its main contribution is to treat cryptographic migration as a distributed governance problem rather than only algorithm replacement or use of a cryptographic provider.

Revalidation strengthened the empirical basis. Baseline and canary operated without HTTP failures in nominal windows; the application chain worked; S4-COMP established a controlled operational comparison; S4-E03 completed 10,000 requests; and S4-E01 recorded mock-vault failure and recovery. The same experiments also delimit the proposal: stress violated LAT-01, failure produced HTTP 500, and observability remained incomplete.

It cannot be concluded that the architecture implements real mTLS, a real KMS, or PQC in the dataplane. Differences in S4-COMP also cannot be attributed to cryptographic effects. These limitations are central to correct interpretation.

Future work should:

- 1 deploy real mTLS and record negotiation and certificates;
- 2 integrate a realistic KMS/HSM through PKCS #11 or an equivalent API;
- 3 execute ML-KEM, ML-DSA, and/or SLH-DSA in the dataplane;
- 4 measure cryptographic overhead using controlled baselines and repetitions;
- 5 automate rollback and test its effectiveness;
- 6 repair Prometheus/Grafana and healthchecks;
- 7 investigate queues, connections, and resources explaining the S4-E03 tail;
- 8 pin image versions and digests;
- 9 repeat experiments with statistical analysis;
- 10 evaluate a cloud or more robust production-like environment.

REFERENCES

- BARKER, Elaine et al. **Considerations for Achieving Crypto Agility: Strategies and Practices**. Gaithersburg: National Institute of Standards and Technology, 2025. NIST CSWP 39. DOI: 10.6028/NIST.CSWP.39.
- BOS, Joppe et al. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM. In: **IEEE European Symposium on Security and Privacy**, 2018. p. 353-367. DOI: 10.1109/EuroSP.2018.00032.
- CBOMKIT PROJECT. **CBOMkit: the essentials for Cryptography Bills of Materials**. 2026. Available at: <https://github.com/cbomkit/cbomkit>. Accessed on: June 11, 2026.
- CHO, Jihoon et al. **Software-Defined Cryptography: A Design Feature of Cryptographic Agility**. arXiv:2404.01808, 2024. Available at: <https://arxiv.org/abs/2404.01808>. Accessed on: June 11, 2026.
- HASAN, Khondokar Fida et al. **A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies**. arXiv:2307.06520, 2023. Available at: <https://arxiv.org/abs/2307.06520>. Accessed on: June 11, 2026.
- IBM RESEARCH. **Cryptographic Bill of Materials Speeds Quantum-Safe Adoption**. 2022. Available at: <https://research.ibm.com/blog/cryptographic-bill-of-materials>. Accessed on: June 11, 2026.
- MICROSOFT. **Cryptographic Service Providers**. 2021. Available at: <https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptographic-service-providers>. Accessed on: June 11, 2026.
- MICROSOFT. **Cryptography API: Next Generation**. 2025. Available at: <https://learn.microsoft.com/en-us/windows/win32/seccng/cng-portal>. Accessed on: June 11, 2026.
- MOSCA, Michele. Cybersecurity in an era with quantum computers: will we be ready? **IEEE Security & Privacy**, v. 16, n. 5, p. 38-41, 2018. DOI: 10.1109/MSP.2018.3761723.
- NÄTHER, Christian et al. **Toward a Common Understanding of Cryptographic Agility: A Systematic Review**. arXiv:2411.08781, 2024. Available at: <https://arxiv.org/abs/2411.08781>. Accessed on: June 11, 2026.
- NATIONAL CYBERSECURITY CENTER OF EXCELLENCE - NCCOE. **Migration to Post-Quantum Cryptography: Quantum Readiness: Cryptographic Discovery**. NIST SP 1800-38B, preliminary draft. Gaithersburg: NIST, 2023a.
- NATIONAL CYBERSECURITY CENTER OF EXCELLENCE - NCCOE. **Migration to Post-Quantum Cryptography: Quantum Readiness: Testing Draft Standards**. NIST SP 1800-38C, preliminary draft. Gaithersburg: NIST, 2023b.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY - NIST. **FIPS 140-3: Security Requirements for Cryptographic Modules**. Gaithersburg: NIST, 2019. DOI: 10.6028/NIST.FIPS.140-3.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY - NIST. **FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard**. Gaithersburg: NIST, 2024a. DOI: 10.6028/NIST.FIPS.203.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY - NIST. **FIPS 204: Module-Lattice-Based Digital Signature Standard**. Gaithersburg: NIST, 2024b. DOI: 10.6028/NIST.FIPS.204.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY - NIST. **FIPS 205: Stateless Hash-Based Digital Signature Standard**. Gaithersburg: NIST, 2024c. DOI: 10.6028/NIST.FIPS.205.

OASIS OPEN. **PKCS #11 Cryptographic Token Interface Base Specification Version 3.1**. 2023. Available at: <https://docs.oasis-open.org/pkcs11/pkcs11-base/v3.1/os/pkcs11-base-v3.1-os.html>. Accessed on: June 11, 2026.

ORACLE. **Java Cryptography Architecture (JCA) Reference Guide**. Java SE 25, 2025. Available at: <https://docs.oracle.com/en/java/javase/25/security/java-cryptography-architecture-jca-reference-guide.html>. Accessed on: June 11, 2026.

OWASP CYCLONEDX. **Cryptography Bill of Materials (CBOM)**. 2026. Available at: <https://cyclonedx.org/capabilities/cbom/>. Accessed on: June 11, 2026.

PODMAN PROJECT. **Podman Documentation**. 2026. Available at: <https://podman.io/docs>. Accessed on: June 11, 2026.

APPENDIX A - EXPERIMENTAL EVIDENCE

The primary package is `revalidation-evidence-2026-06-10.tgz`, located at the repository root. Once extracted, the `revalidation/` directory contains:

- `summary-results.md`;
- S4-T01 and S4-T02 k6 summaries and outputs;
- the S4-T03 `app-chain.json` response;
- S4-COMP summaries and comparison;
- S4-E03 summary, output, and report;
- S4-E01 captures before, during, and after failure;
- runtime captures.

An organized copy is also available under `docs/revalidation/`.

APPENDIX B - REPRODUCTION COMMANDS

```
# Validate composition
podman compose -f lab/topology/docker-compose.yml config

# Build and start
podman compose -f lab/topology/docker-compose.yml up -d --build

# Check nodes
podman compose -f lab/topology/docker-compose.yml ps

# Run scenarios through the laboratory controller
lab/topology/scripts/labctl.sh s4-t01
lab/topology/scripts/labctl.sh s4-t02
lab/topology/scripts/labctl.sh s4-e04

# Stress example
k6 run lab/topology/k6/stress-10000.js
```

Exact revalidation commands on the server should be checked against the scripts and environment paths before a new run.

APPENDIX C - ARTIFACT STRUCTURE

- code/
 - cbom_gateway.py
 - cbomkit_cli.py
 - samples/
- lab/topology/
 - docker-compose.yml
 - k6/
 - scripts/
- docs/module14/
- docs/module16/
- docs/revalidation/
- revalidation-evidence-2026-06-10.tgz
- paper/
- tcc/

APPENDIX D - PENDING ITEMS FOR THE FINAL VERSION

- 1 Confirm the full name, order, and role of all authors.
- 2 Confirm the degree program, advisor, co-advisor, and committee members.
- 3 Request the cataloging record and approval page.
- 4 Decide whether acknowledgments should be retained.
- 5 Update the table of contents and list of tables in Word after final pagination.
- 6 Validate the official English institutional name for the cover.
- 7 Reexecute S4-E04 on the server and archive the final log if it is presented as server evidence.
- 8 Review references under the ABNT edition adopted by Inteli and validate preprint references.